

Практическое задание №4

Helm, CRD & RBAC

- Переработайте развертывание из практических заданий 2 и 3 с обычных манифестов на Helm
- GatewayApi тоже должен ставиться через Helm
- Замените самостоятельно развернутый PostgreSQL на CloudNativePG оператор
- Добавьте самоподписанные сертификаты, используя CertManager оператор

Введение

Цель работы: Данная работа посвящена переходу от ручного управления ресурсами (kubectl apply) к декларативному управлению жизненным циклом приложения с помощью пакетного менеджера Helm. В рамках работы была спроектирована и развернута микросервисная среда, включающая базу данных под управлением оператора, управление доступом на основе ролей (RBAC) и современную систему маршрутизации трафика (Gateway API).

Целью данной работы является проектирование и развертывание отказоустойчивой микросервисной архитектуры в кластере Kubernetes, обеспечивающей:

- **Автоматизацию развертывания:** Создание Helm-чарта для стандартизации процесса установки, обновления и отката приложения.
- **Безопасность инфраструктуры:** Реализацию принципа минимальных привилегий (Least Privilege) через настройку ServiceAccount и RBAC (Role-Based Access Control).
- **Управление данными:** Развертывание и настройку высокодоступной базы данных с использованием оператора CloudNativePG.
- **Современную маршрутизацию:** Организацию входящего трафика с использованием Gateway API для эффективного управления сетевыми запросами к микросервису.

Ход работы.

Развертывание системы было разделено на этапы, каждый из которых решал конкретную инженерную задачу.

Создание структуры Helm-чарта

На первом этапе был инициализирован Helm-чарт microshop. Была разработана структура шаблонов (templates/), обеспечивающая разделение логики (Deployment, Service, ServiceAccount, RBAC) и конфигурации (values.yaml). Это позволило параметризовать окружение: вынести реплики, образы и переменные окружения во внешний файл.

Согласно принципу минимальных привилегий, был создан выделенный ServiceAccount (microshop-sa). Для обеспечения доступа приложения к объектам кластера был настроен RoleBinding, связывающий ServiceAccount с Role, ограничивающей права только операциями get, list, watch для pods и configmaps.

Для работы с данными был развернут объект Cluster через оператор CloudNativePG. В ходе работы был решен конфликт владения ресурсом: с помощью меток (labels) и аннотаций (annotations) объект Cluster был передан под управление Helm-релиза, что обеспечило согласованность конфигурации.

Взамен классического Ingress был использован Gateway API. Был настроен объект HTTPRoute, который описывает правила маршрутизации трафика от Gateway (eg) к сервису приложения. В процессе настройки были решены проблемы с валидацией метаданных (apiVersion/kind) и привязкой к существующему шлюзу (parentRefs).

Финальным этапом стала сборка всех манифестов в единый релиз. Использование команды helm install позволило автоматизировать создание всей цепочки объектов.

Postgres-cluster.yaml:

```
1  apiVersion: postgresql.cnpg.io/v1
2  kind: Cluster
3  metadata:
4    name: postgres-cluster
5  spec:
6    instances: 1
7    primaryUpdateStrategy: unsupervised
8    storage:
9    size: 16i
```

Rbac.yaml

```
1  apiVersion: rbac.authorization.k8s.io/v1
2  kind: Role
3  metadata:
4    name: microshop-role
5  rules:
6    - apiGroups: [""]
7      resources: ["pods", "configmaps"]
8      verbs: ["get", "list", "watch"]
9  ---
10 apiVersion: rbac.authorization.k8s.io/v1
11 kind: RoleBinding
12 metadata:
13   name: microshop-rolebinding
14 subjects:
15   - kind: ServiceAccount
16     name: microshop-sa
17 roleRef:
18   kind: Role
19   name: microshop-role
20 apiGroup: rbac.authorization.k8s.io
```

Values.yaml:

```
1  replicaCount: 1
2
3  image:
4    repository: vegetarianz/pythonrest
5    pullPolicy: IfNotPresent
6    tag: "latest"
7
8  nameOverride: "microshop"
9
10 env:
11   name: "Nikita"
12   dbHost: "postgres-cluster-rw"
13
14 serviceAccount:
15   create: true
16   name: "microshop-sa"
17
18 autoscaling:
19   enabled: false
20
21 service:
22   type: ClusterIP
23   port: 80
24
25 httpRoute:
26   enabled: true
```

Sertificate.yaml:

```
1  apiVersion: cert-manager.io/v1
2  kind: Issuer
3  metadata:
4    name: selfsigned-issuer
5  spec:
6    selfSigned: {}
7  ---
8  apiVersion: cert-manager.io/v1
9  kind: Certificate
10 metadata:
11   name: microshop-tls
12 spec:
13   secretName: microshop-tls-secret
14   issuerRef:
15     name: selfsigned-issuer
16   dnsNames:
17     - "microshop.local"
```

Serviceaccount.yaml:

```
1  apiVersion: v1
2  kind: ServiceAccount
3  metadata:
4    name: microshop-sa
5    labels:
6      {{- include "microshop.labels" . | nindent 4 }}
7  automountServiceAccountToken: true
```

Проверка состояния существующей инфраструктуры и определение имен ресурсов для дальнейшей интеграции:

- `kubectl get gateways` — получение списка шлюзов для настройки маршрутизации.
- `kubectl get gatewayclasses` — определение доступных классов шлюзов.

Для устранения конфликтов «владения» ресурсами, которые были созданы до развертывания через Helm, были выполнены команды «усыновления» (adoption):

- `kubectl label cluster postgres-cluster app.kubernetes.io/managed-by=Helm --overwrite` — присвоение метки управления.
- `kubectl annotate cluster postgres-cluster meta.helm.sh/release-name=my-microshop --overwrite` — привязка к Helm-релизу.
- `kubectl annotate cluster postgres-cluster meta.helm.sh/release-namespace=default --overwrite` — привязка к пространству имен.

Команды для развертывания, проверки и очистки ресурсов:

- `helm install my-microshop ./microshop` — установка приложения из чарта.
- `helm uninstall my-microshop` — удаление релиза для сброса состояния (при возникновении ошибок).
- `helm list --all-namespaces` — проверка установленных релизов во всех пространствах имен.
- `helm template my-microshop ./microshop --debug` — отладка шаблонов (использовалась для поиска ошибок в манифестах).

Финальные команды для подтверждения корректности работы системы:

- `kubectl get pods` — проверка статуса подов приложения.
- `kubectl get httproute` — проверка успешной настройки маршрутизации.
- `kubectl logs -l app.kubernetes.io/instance=my-microshop` — просмотр логов для подтверждения успешного подключения к базе данных.

Заключение.

В ходе выполнения лабораторной работы была успешно решена задача перевода микросервисной инфраструктуры с ручного развертывания на декларативную модель управления с использованием Helm.

Основные результаты работы:

1. **Автоматизация и стандартизация:** Благодаря разработке Helm-чарта процесс развертывания приложения стал воспроизводимым. Теперь для развертывания всей экосистемы — от базы данных до маршрутизации — достаточно одной команды, что минимизирует риск «человеческого фактора» и конфигурационных ошибок.
2. **Безопасность:** Внедрение механизмов RBAC позволило реализовать принцип минимальных привилегий. Приложение теперь работает в изолированном контексте ServiceAccount, что значительно повышает защищенность кластера по сравнению с использованием учетных записей по умолчанию.
3. **Отказоустойчивость:** Использование оператора CloudNativePG для управления базой данных обеспечивает высокую доступность данных и автоматизированное восстановление ресурсов при сбоях.
4. **Современные сетевые стандарты:** Переход на Gateway API позволил настроить гибкую и масштабируемую маршрутизацию трафика, что является более актуальным и производительным решением по сравнению с классическими Ingress контроллерами.

Выводы

Данная работа продемонстрировала важность грамотного управления инфраструктурой как кодом (IaC). В процессе реализации были успешно решены сложные технические задачи: интеграция существующих объектов в Helm-релиз, отладка шаблонизации и устранение ошибок валидации API.

Полученные навыки работы с Kubernetes-операторами, управлением правами доступа и сетевыми абстракциями являются ключевыми компетенциями для инженера DevOps. Реализованная инфраструктура готова к дальнейшему масштабированию и интеграции в CI/CD конвейеры, что является логическим продолжением данной работы.